OpenMP Parallelization of a Feedforward Neural Network

Implementation and Experimental Analysis

Parallel Programming Laboratory
Universitat Autònoma de Barcelona

Abstract

We investigate OpenMP parallelization of the supplied feedforward neural network, progressing from a corrected sequential baseline to loop-based and task-based implementations. The report follows our implementation decisions and their measured outcomes. Evaluation focuses on execution time, speedup, efficiency, and strong scaling, with brief checks of neural-network behavior. Findings and conclusions will be added after the experiments.

1 Experimental approach

We parallelize the supplied C program and evaluate its performance as the thread count increases [1]. After examining dependencies and establishing a corrected sequential baseline, we develop loop-based parallelism, then investigate a task-based alternative. The assignment permits either approach; exploring both is our choice. Comparisons preserve the workload, compiler settings, and timing boundaries. The original source remains recoverable in Git.

1.1 Measurement and scaling

We briefly check successful execution, finite losses, and consistent recognition results. Evaluation focuses on parallel performance. We also examine shared data and synchronization to ensure valid execution.

Following the course notation [3], let N denote problem size, p the thread count, $T(N,1)$ our measured sequential time, and $T(N,p)$ the parallel time. Then

$$S(N,p) = \frac{T(N,1)}{T(N,p)}, \tag{1}$$

$$E(N,p) = \frac{S(N,p)}{p}. \tag{2}$$

We use our own sequential baseline on the same hardware and workload. OpenMP runs with one thread provide an additional overhead comparison; published timings remain a separate reference.

We evaluate **strong scaling**: keep the dataset, network, batch size, and epochs fixed and vary $p \in \{1, 2, 4, 6, 12\}$. We will plot time, speedup, and efficiency against thread count. Ideally, $T(N,p) = T(N,1)/p$, $S(N,p) = p$, and $E(N,p) = 1$. These curves show where additional threads stop helping. **Weak scaling** grows the workload with the thread count and is outside this fixed-workload experiment. We will report repeated timings and variability, and distinguish measured explanations from hypotheses.

1.2 Course references

Table 1 gives indicative course timings for an optimized OpenMP implementation [2]. We have access to Wilma and Aolin, but the reference source code and exact timing boundaries are unavailable, limiting direct comparisons.

Table 1: Course reference execution times in seconds (10 epochs).

Threads	Wilma `nodo.q`	Wilma `new-nodo.q`	Aolin `aolin.q`
Sequential	40.737795	12.095956	11.235670
2	22.542747	6.873421	6.767024
4	11.730357	4.296542	3.658490
6	8.308845	3.474935	2.502528
12	5.263955	2.555904	2.380782

2 Understanding the sequential program

2.1 Execution flow

In `main()`, configuration and initialization precede the timer; training and testing follow, before the timer stops and final cleanup begins [5]. The measured interval includes training, testing, dataset loading, console output, and input-data cleanup. We preserve this boundary. Testing uses the trained weights, so training and testing must remain ordered.

The default network has layers $1024 \to 500 \to 210 \to 135 \to 10$, with 10 training epochs and batches of 50 patterns. The supplied batch loop processes 38 complete batches per epoch: 1,900 of the 1,934 training

patterns, after shuffling. We preserve this workload, including omission of the last 34 patterns. Testing uses all 946 test patterns.

2.2 Baseline corrections

Before benchmarking, we corrected four implementation issues:

- **Image loading:** a bounded row buffer prevents the string terminator from overflowing the pixel array.
- **Random-number generation:** unsigned 32-bit arithmetic avoids signed overflow while preserving the usual wraparound sequence.
- **Dataset loading:** training targets are initialized only when loading training data, preventing unintended writes during testing.
- **Bias handling:** gradients are reset and applied to hidden and output biases, excluding the input layer.

Both sequential and parallel versions use these corrections. Updating the output biases can change recognition results.

3 Sequential baseline

After a preliminary execution, we collected five sequential runs on `clus07.hpc.local`, in Wilma's `new-nodo.q` partition. The node reports an Intel Xeon Gold 5416S, two sockets with 16 physical cores each, and two hardware threads per core (32 physical cores, 64 logical CPUs). We used GCC 13.2.1 with `-Ofast`, linking `-lm`, and kept the configuration and executable unchanged between runs.

Table 2: Sequential application timings on `clus07`, five runs. Standard deviation is the sample standard deviation.

Statistic	Time (s)
Mean	12.343398
Standard deviation	0.108668
Minimum	12.231889
Maximum	12.474166
Course sequential reference	12.095956

The standard deviation is 0.88% of the mean. All five runs produced the same printed epoch losses, a final loss of 0.055671, and 916 correct classifications out of 946. We use the mean application time, 12.343398 s, as $T(N, 1)$ for comparisons on this node and configuration; `perf`'s process-wide elapsed time is not used. The individual runs and environment are retained in the measurement record. Our mean exceeds the course reference by 2.05%; the cause is undetermined. Up to 12 threads fit on physical cores; placement will be checked.

4 Loop-based OpenMP

4.1 Forward propagation

We added `parallel for schedule(static)` to the neuron loop in `forward_prop()` [4]. Each iteration reads the preceding layer's activations, weights, and bias, and writes distinct `z[j]` and `actv[j]` elements. Layers remain ordered, with loop completion ensuring that all activations are available to the next layer. The inner sum remains sequential, retaining its accumulation order. Static scheduling suits the equal-length sums. Repeated parallel regions and synchronization may limit gains in small layers; this requires measurement.

4.2 Backpropagation and shared sums

Different neurons in the supplied `back_prop()` add to the same `dactv[k]`, making direct neuron-loop parallelization unsafe. We separated the work: parallel neuron iterations compute `dz`, bias gradients, and distinct weight-gradient rows; parallel `k` iterations then accumulate individual `dactv[k]` values in the original neuron order. Loop completion enforces dependencies; input-layer derivatives are omitted as before. Each element has one writer per phase. Patterns remain sequential because they share activation buffers and batch gradients.

4.3 Gradient reset and weight updates

Next, we will distribute weight-gradient resets across array elements and weight updates across independent neuron pairs using `collapse(2)`. Both use static scheduling and distinct writes, without reductions. The small bias loops remain sequential to limit parallel-region overhead. Gradient reset, batch accumulation, and parameter updates remain ordered. Benchmarking follows completion of these changes and OpenMP build setup.

References

[1] Universitat Autònoma de Barcelona, *Parallel Programming Laboratory Project*, 2026–2027, Sections III–IV. Repository copy.

[2] Universitat Autònoma de Barcelona, *FeedForward Neural Networks: Reference Time – SEQ + OpenMP*, October 2026. Repository copy.

[3] Universitat Autònoma de Barcelona, *Lecture 3: Introduction to Parallelism*, slides 20–22, 26–30.

[4] Universitat Autònoma de Barcelona, *Lecture 4: OpenMP*, slides 25–37, 44–53.

[5] Supplied sequential implementation, `main.c`, functions `main()`, `train_neural_net()`, and `test_nn()`.